

A
PROJECT REPORT
ON
Safar Ai (Travel Trip Planner)

SUBMITTED IN THE PARTIAL FULFILLMENT OF THE REQUIREMENT FOR OF

MASTER OF COMPUTER APPLICATION

SUBMITTED BY

Achal Abasaheb Pawar : FY-MCA 03

Bhushan Sunil Jawale : FY-MCA 19

Harish Chandrakant Bhamre : FY-MCA 32

UNDER THE GUIDANCE OF

Dr. Priya Chaudhari



G. H. RAISONI COLLEGE OF ENGINEERING AND MANAGEMENT

(An Autonomous Institute affiliated to SPPU)

WAGHOLI, PUNE : 412207

SAVITRIBAI PHULE PUNE UNIVERSITY

2025 - 2026



CERTIFICATE

This is to certify that the project report entitled

Safar Ai (Travel Trip Planner)

SUBMITTED BY

Achal Abasaheb Pawar : FY-MCA 03

Bhushan Sunil Jawale : FY-MCA 19

Harish Chandrakant Bhamre : FY-MCA 32

Are bonafide students of this institute and the work has been carried out by them under the supervision of **Dr. Priya Chaudhari** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Master of Computer Application**.

Dr. Priya Chaudhari

Internal Guide

Dr. Priya Chaudhari

HOD, MCA

External

Asst. Prof. Kishor Markad

Project Coordinator

Dr. N. B. Hulle

Director

Place: Pune

Date: _____

Acknowledgement

We are profoundly grateful to **Dr. Priya Chaudhari** for his/her expert guidance and continuous encouragement throughout to see that this project rights its target since its commencement to its completion.

We would like to express our deepest appreciation towards **Dr. N. B. Hulle**, Director, G. H. Rasoni College of Engineering and Management, Pune, **Dr. Priya Chaudhari**, Head of Department, Computer Engineering and **Asst. Prof. Kishor Markad**, Project Coordinator whose invaluable guidance supported us in completing this project.

At last, we must express our sincere heartfelt gratitude to all the staff members of the Computer Engineering Department who helped us directly or indirectly during this course of work.

Achal Abasaheb Pawar _____

Bhushan Sunil Jawale _____

Harish Chandrakant Bhamre _____

Table of Content

Chapter No		Details	Page No.
1		Introduction	1
	1.1	Abstract	3
	1.2	Existing System and Need for System	4
	1.3	Scope of System	5
	1.4	Operating Environment (Technology) – Hardware, Software and Database	6
2		Analysis and Design	
	2.1	List of Modules	9
	2.2	Project Planning and Scheduling	10
	2.3	Entity Relationship Diagram (ERD)	11
	2.4	Table Structure	11
	2.5	Use Case Diagrams	12
	2.6	Class Diagram	12
	2.7	Activity Diagram	13
	2.8	Sequence Diagram	13
	2.9	Deployment Diagram	14
	2.10	Module Hierarchy Diagram	14
	2.11	Input and Output Screens (Screens must have valid data. All reports must have at-least 5 valid records.)	15-17
3		Coding	
	3.1	Complete Code of Project or one Module	18
4		Testing	
	4.1	Test Strategy	21
	4.2	Unit Test Plan	22
	4.3	Acceptance Test Plan	24
	4.4	Test Case / Test Script	25
5		Limitations of Proposed System	30
6		Proposed Enhancements	32
7		Conclusion	34
8		Bibliography	35

1. Introduction

Safar AI is an intelligent, AI-powered travel planning web application designed specifically for Indian travellers. The name "Safar" (meaning "journey" in Hindi/Urdu) reflects the platform's core purpose — to make every journey seamless, personalised, and effortless. The application serves as a one-stop travel companion that allows users to discover destinations across India, plan complete multi-day itineraries, search for flights and hotels, and interact with a conversational AI travel assistant — all within a single mobile-first web interface. By integrating cutting-edge AI models (Google Gemini and Groq/LLaMA), Safar AI generates real-time, context-aware travel plans tailored to each user's budget, interests, travel style, and home location, bridging the gap between traditional travel booking platforms and modern AI capabilities.

Travel planning in India is uniquely complex. The country spans 28 states and 8 union territories, offering an almost overwhelming diversity of landscapes — from the snow-capped Himalayas in the north to the tropical backwaters of Kerala in the south. A traveller planning a trip to Rajasthan must simultaneously consult multiple platforms for flights, hotels, attractions, and restaurants, manually stitching together a coherent plan from dozens of disconnected data points. Most platforms offer the same generic recommendations regardless of whether the user is a solo backpacker on a ₹5,000 budget or a family of four celebrating an anniversary. Static travel guides cannot account for seasonal variations, local festivals, or weather disruptions, and most premium platforms are English-first, creating an accessibility gap that excludes a significant portion of India's growing middle-class travel market. Safar AI was conceived as a direct response to all of these pain points — not merely as another booking engine, but as an intelligent planning ecosystem that truly understands the Indian traveller's unique needs, constraints, and aspirations.

The vision behind Safar AI is to become India's most trusted AI-powered travel companion, democratising intelligent travel planning by making personalised, AI-generated itineraries accessible to every Indian traveller through a simple, intuitive, and conversational interface. The design philosophy rests on three foundational pillars — radical personalisation, conversational accessibility, and end-to-end integration. Every itinerary and recommendation is tailored to the individual user, considering their home city, budget, travel style, group composition, and trip duration. Rather than forcing users through complex forms and filter menus, the platform offers a natural language interface through which users simply describe what they want and let the AI handle the rest, making Safar AI the only platform a traveller needs from initial inspiration to final itinerary.

The technology foundation is built on a modern, scalable stack. The frontend uses React.js with Tailwind CSS for a clean, mobile-first experience, while the backend runs on Node.js with Express.js for efficient API orchestration. Google Gemini powers the itinerary generation engine with its ability to understand complex prompts and produce structured, detailed responses, while Groq's LLaMA model drives the conversational assistant with ultra-fast inference for real-time interaction. Firebase handles authentication and data storage through Firestore, persisting user profiles, saved itineraries, and conversation histories.

Integrations with the Google Places API, aviation data APIs, and weather services further enrich recommendations with real-world data.

The flagship feature is AI-generated itineraries that include day-by-day activities, meal recommendations, local transport options, estimated costs, and accommodation suggestions — all dynamically constructed from the user's specific inputs. Complementing this is a conversational AI assistant that answers travel questions in natural language, maintains conversation context, and understands the nuances of Indian geography, culture, cuisine, and transport infrastructure. The platform also offers destination discovery for users in the inspiration phase, integrated flight and hotel search within the planning workflow, and user profiles that learn preferences over time to deliver increasingly personalised recommendations.

India's domestic tourism sector records over 1.5 billion visits annually, yet existing platforms remain primarily transactional, leaving the planning phase largely unaddressed. With smartphone penetration crossing 700 million users and Indian consumers increasingly comfortable with AI-powered experiences, the market is ready for an intelligent planning companion. No existing platform combines deep India-specific knowledge, genuine AI personalisation, and end-to-end planning in the way Safar AI does — positioning it not as a feature addition to an existing category, but as the creation of an entirely new one.

Beyond commercial potential, Safar AI carries meaningful social impact — promoting domestic tourism, democratising travel planning for Tier 2 and Tier 3 city travellers, encouraging sustainable tourism patterns, and embedding cultural intelligence into every itinerary. Safar AI is more than a technology product — it is a travel companion that understands India, understands Indian travellers, and is committed to making every journey as seamless, enriching, and memorable as possible. That is the promise of Safar AI. That is the meaning of Safar.

1.1 Abstract

Safar AI is a full-stack, AI-driven travel planning platform built specifically for the Indian travel market, combining modern web technologies with powerful large language models to deliver a seamless, personalised travel planning experience. The frontend is developed using HTML5, CSS3, and Vanilla JavaScript with ES6 Modules, structured as a Single Page Application with client-side routing that ensures smooth, app-like navigation without page reloads. The backend is a lightweight Python Flask REST API that serves as an AI orchestration layer, managing communication between the frontend and the various AI and third-party services that power the platform's core features.

The intelligence behind Safar AI comes from two complementary AI systems. Google Gemini 1.5 Flash handles itinerary generation and destination recommendations, leveraging its ability to process complex, multi-part prompts and return structured, detailed travel plans in real time. Groq's LLaMA 3.1 model powers the flight search, hotel search, and conversational chat assistant, taking advantage of Groq's ultra-fast inference speeds to deliver responses that feel immediate and natural. Together, these two models cover the full spectrum of the platform's AI needs — from deep, structured planning to quick, conversational interaction.

User authentication and data persistence are handled entirely through Firebase. Firebase Authentication supports both Email/Password and Google OAuth login, providing users with a secure and frictionless sign-in experience. Cloud Firestore serves as the primary database, storing user profiles, travel preferences, wishlists, and saved trips in a flexible, real-time NoSQL structure that scales effortlessly with user growth. Dynamic destination imagery is fetched from the Pexels API, enriching the visual experience of the platform and ensuring that destination cards and itinerary pages are accompanied by high-quality, contextually relevant photography.

The platform is built around several key features that together form a complete travel planning workflow. A multi-step onboarding flow captures each user's travel preferences — including their home city, preferred travel style, budget range, and interests — at the point of registration, enabling the AI to deliver personalised recommendations from the very first interaction. The AI trip planner streams itinerary generation in real time, allowing users to watch their personalised day-by-day travel plan appear progressively rather than waiting for a complete response, creating a more engaging and responsive experience. Flight and hotel search are powered by LLM-generated realistic data, surfacing relevant options within the planning interface without requiring users to navigate to external booking platforms. A destination explorer with category filtering and search allows users in the inspiration phase to browse and discover places that match their interests, while a wishlist system lets them save and revisit destinations they want to explore in the future.

1.2 Existing System and Need for System

The Indian travel technology landscape is currently dominated by platforms that, while functional, fall significantly short of offering a truly intelligent and integrated travel planning experience. MakeMyTrip, Goibibo, and Yatra are primarily transactional booking platforms focused on flight and hotel listings, offering no AI-driven personalisation or itinerary planning capabilities. Google Travel provides basic trip organisation but lacks deep itinerary generation or conversational assistance. TripAdvisor offers reviews and suggestions but requires users to manually piece together a trip plan from scattered information. General-purpose AI tools like ChatGPT can generate travel advice but have no integration with user profiles, booking data, or real-time travel context, making them useful only as a starting point rather than a complete planning solution.

The core limitation shared across all existing systems is fragmentation. No single platform combines destination discovery, AI itinerary generation, transport search, hotel search, and a conversational assistant in one place. Existing booking platforms are transactional, not advisory — they present options but do not plan a trip for the user. A traveller must visit multiple platforms, cross-reference information manually, and invest significant time and effort before arriving at a coherent travel plan. General-purpose AI tools, while capable of generating travel suggestions, lack the personalisation that comes from knowing the user's home location, budget constraints, travel style, and past preferences. No existing Indian travel application offers real-time streaming AI itinerary generation with day-by-day activity breakdowns, meal suggestions, and cost estimates — the kind of structured, actionable output that transforms a vague travel idea into a ready-to-execute plan.

Safar AI was built specifically to address these gaps. By combining AI intelligence with practical travel utility in a single mobile-first platform, it eliminates the need for users to juggle multiple tools and manually consolidate information. Every recommendation within Safar AI is personalised based on the user's saved preferences — including their home city, budget range, travel interests, and preferred trip style — ensuring that the platform delivers relevant, contextually appropriate suggestions from the very first interaction. The AI trip planner generates complete, structured itineraries covering transport, accommodation, daily activities, meal suggestions, and cost estimates in a single, streamlined workflow. The conversational assistant retains full chat history, enabling context-aware travel advice that builds naturally across a conversation rather than treating each question in isolation.

Critically, Safar AI is designed with the Indian travel market at its core. INR pricing, Indian airlines, Indian destinations, and India-centric AI prompts ensure that the platform speaks directly to the needs and expectations of Indian travellers — something no existing platform, domestic or global, currently delivers with the depth and intelligence that Safar AI provides.

1.3 Scope of System

The scope of Safar AI covers the following functional areas:

User Management

- User registration and login via Email/Password and Google OAuth (Firebase Auth).
- Multi-step onboarding to capture home city, travel interests, budget preference, and trip style.
- User profile management and preference updates.

Destination Discovery

- Home screen with popular Indian destinations and dynamic imagery.
- Explore page with 20+ curated destinations filterable by category (Beaches, Mountains, Heritage, Nature, Spiritual, Adventure, Hill Stations).
- Search functionality within the explore page.
- AI-powered nearby destination recommendations based on user's home location.

AI Trip Planner

- Input: source city, destination city, number of days (1–30), and budget tier (Budget / Balanced / Premium).
- Real-time streaming itinerary generation via Google Gemini.
- Output: flight options, train options, cab/local transport, hotel recommendations, and a full day-by-day itinerary with timed activities, descriptions, durations, costs, and meal suggestions.
- Save generated trips to Firebase Firestore.

Flight Search

- Search flights by origin, destination, date, passengers, and cabin class.
- AI-generated realistic flight options using Indian airlines (IndiGo, Air India, SpiceJet, Vistara, AirAsia India) via Groq/LLaMA.

Hotel Search

- Search hotels by city, budget tier, guests, rooms, check-in/check-out dates, and property type.
- AI-generated realistic hotel listings with amenities, ratings, and INR pricing via Groq/LLaMA.

AI Chat Assistant

- Conversational travel assistant powered by Groq (LLaMA 3.1) with Gemini as fallback.
- Maintains conversation history (last 10 messages) for context-aware responses.
- Pre-built suggestion chips for common travel queries.
- Supports markdown-like formatting in responses (bold, bullet points, code).

Wishlist

- Users can save and manage a wishlist of desired destinations stored in Firebase Firestore.

Deals Page

- Dedicated section for travel deals and offers.

1.4 Operating Environment (Technology) - Hardware, Software, and Database

1. Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 / AMD Ryzen 3 (2.0 GHz)	Intel Core i5 / AMD Ryzen 5 (2.5 GHz+)
RAM	4 GB	8 GB or more
Storage	500 MB free disk space	2 GB SSD
Network	Broadband internet (required for AI API calls)	High-speed broadband (10 Mbps+)
Display	1280 × 720 resolution	1920 × 1080 Full HD
Operating System	Windows 10 / macOS 10.15 / Ubuntu 20.04	Windows 11 / macOS 13+ / Ubuntu 22.04

2. Client / End-User Device Requirements

Device Type	Requirement
Smartphone	Android 8.0+ or iOS 13+ with a modern browser
Tablet	Any tablet with Chrome, Safari, or Firefox
Desktop / Laptop	Any machine with a modern web browser
Screen Size	Optimised for 360px–430px (mobile-first), scales to desktop
Internet	Active internet connection (for AI API calls, Firebase, and image loading)

3. Software Requirements (Frontend Technologies)

Technology	Version	Role
HTML5	Latest	Application shell and SPA structure (index.html)
CSS3	Latest	Complete design system, animations, responsive layout (styles/main.css)
JavaScript (ES6 Modules)	ES2020+	All UI logic, routing, API calls, page rendering
Vanilla JS (No Framework)	—	SPA built without React/Vue/Angular — pure JS modules

4. Backend Technologies

Technology	Version	Role
Python	3.8+	Backend runtime language
Flask	3.0.0	Lightweight REST API framework
Flask-CORS	4.0.0	Enables Cross-Origin Resource Sharing for frontend-backend communication
python-dotenv	1.0.0	Loads API keys from .env file into environment variables
Requests	2.31.0	HTTP client library for calling Gemini and Groq APIs

5. AI / LLM Services (External APIs)

Service	Model	API Endpoint	Usage in Project
Google Gemini	Gemini 1.5 Flash (gemini-3-flash-preview)	generativelanguage.googleapis.com/v1beta	Itinerary generation (streaming), destination recommendations, nearby places
Groq API	LLaMA 3.1 8B Instant	api.groq.com/openai/v1/chat/completions	Flight search, hotel search, chat assistant (primary — faster response)
Gemini (Fallback)	Gemini 1.5 Flash	Same as above	Chat assistant fallback when Groq is unavailable or fails

6. Database (Firebase Cloud Firestore (NoSQL))

Property	Detail
Type	NoSQL Document Database
Provider	Google Firebase
SDK Version	Firebase JS SDK v10.12.2
Access	Client-side (frontend) via Firebase SDK — no backend needed for DB operations
Security	Firebase Security Rules control read/write access per authenticated user
Real-time	Supports real-time listeners (used for auth state via <code>onAuthStateChanged</code>)

2. Analysis and Design

2.1 List of Modules:

The system is divided into the following functional modules:

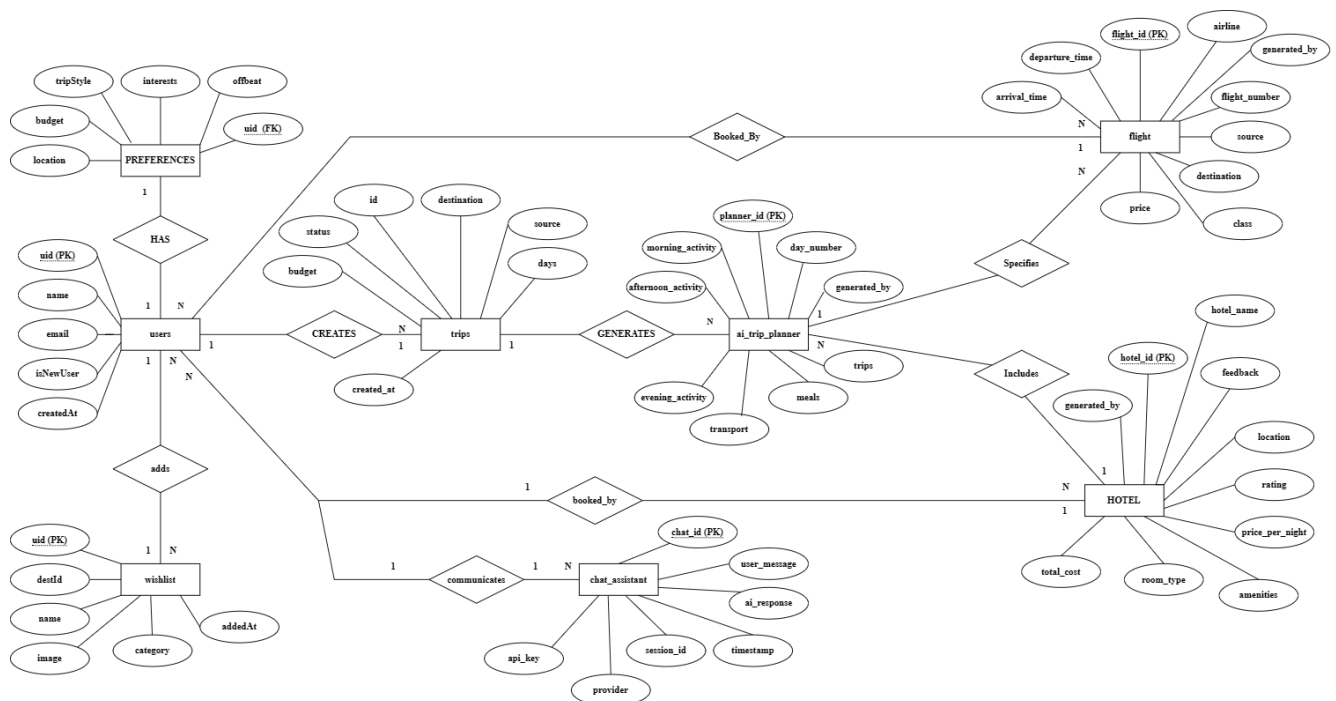
#	Module	Description
1	Authentication Module	User registration, login (Email/Password + Google OAuth), session management via Firebase Auth
2	Onboarding Module	Multi-step preference capture — home city, interests, budget, trip style, offbeat toggle
3	Home / Dashboard Module	Landing page with hero banner, quick actions, popular destinations grid with dynamic images
4	Trip Planner Module	AI itinerary generation with streaming — flights, trains, cabs, hotels, day-by-day activities
5	Flight Search Module	AI-powered flight search by route, date, passengers, and cabin class using Groq/LLaMA
6	Hotel Search Module	AI-powered hotel search by city, budget, guests, rooms, and property type using Groq/LLaMA
7	Explore Module	Destination browser with category filters (Beaches, Mountains, Heritage, etc.) and search
8	AI Chat Assistant Module	Conversational travel assistant with chat history, suggestion chips, and markdown formatting
9	Wishlist Module	Save and manage favourite destinations, persisted in Firestore
10	Profile Module	User profile, travel badges, preference editing, stats display, and logout
11	Deals Module	Travel deals and offers display page
12	Backend API Module	Flask REST API — AI orchestration layer connecting frontend to Gemini and Groq APIs
13	Router Module	Client-side SPA router handling navigation without page reloads
14	Auth Service Module	Firebase Auth wrapper — signup, login, Google OAuth, auth state watching

2.2 Project Planning and Scheduling

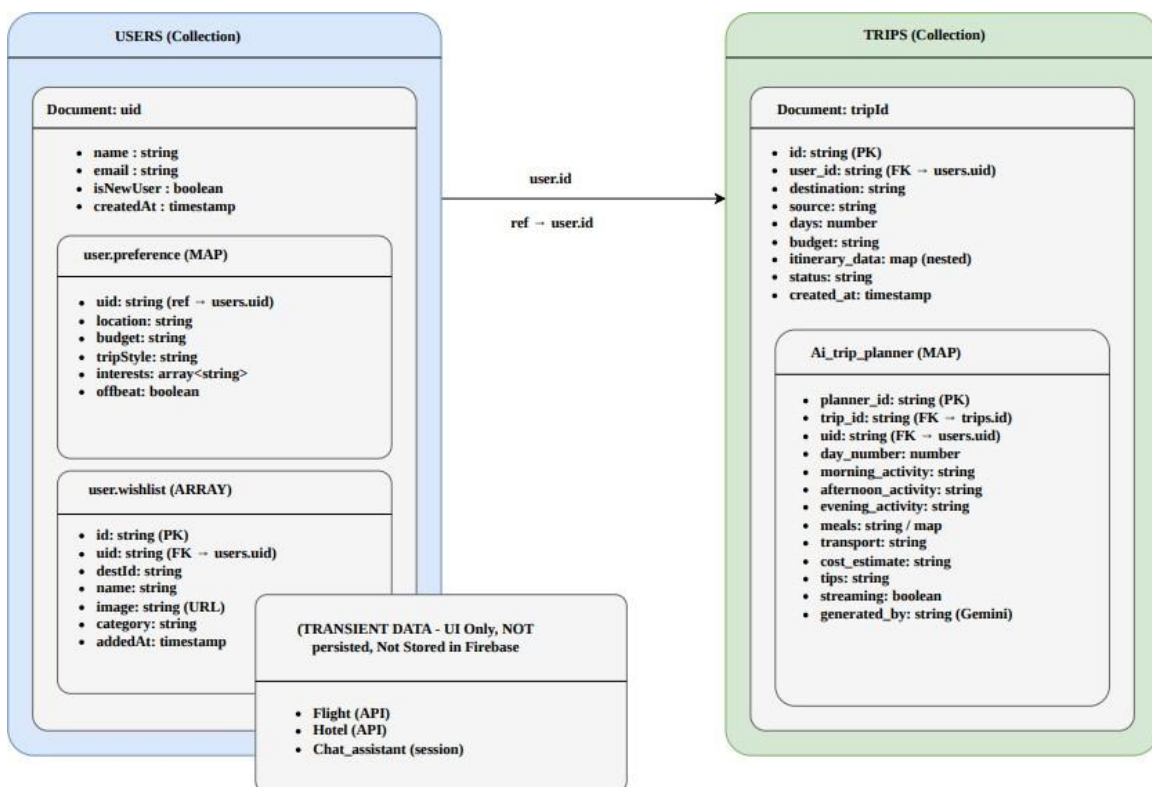
Project Phases and Timeline

Phase	Tasks	Duration
Phase 1 — Requirements & Planning	Requirement gathering, technology selection, system architecture design	Week 1
Phase 2 — UI/UX Design	Wireframing all screens, design system (colors, typography, components), CSS framework	Week 2
Phase 3 — Frontend Development	SPA setup, router, auth pages, onboarding, home, explore, wishlist	Week 3–4
Phase 4 — Backend Development	Flask API setup, Gemini integration, Groq integration, itinerary streaming endpoint	Week 5
Phase 5 — AI Feature Integration	Trip planner, flight search, hotel search, chat assistant, recommendations API	Week 6
Phase 6 — Firebase Integration	Firebase Auth, Firestore schema, user profile, wishlist, trip saving	Week 7
Phase 7 — Testing & Debugging	Unit testing, API testing, UI testing, error handling, fallback mechanisms	Week 8
Phase 8 — Deployment & Documentation	Local deployment scripts, README, setup guide, final documentation	Week 9

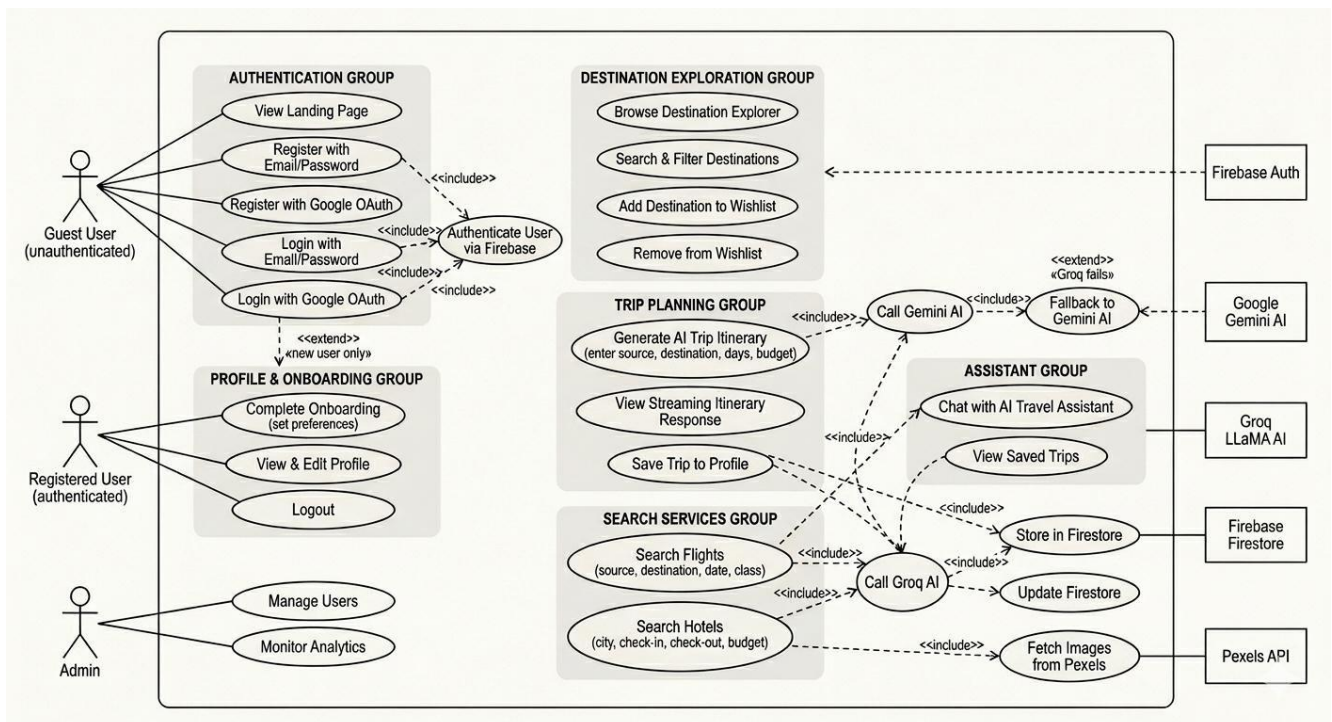
2.3 Entity Relationship Diagram (ERD) :



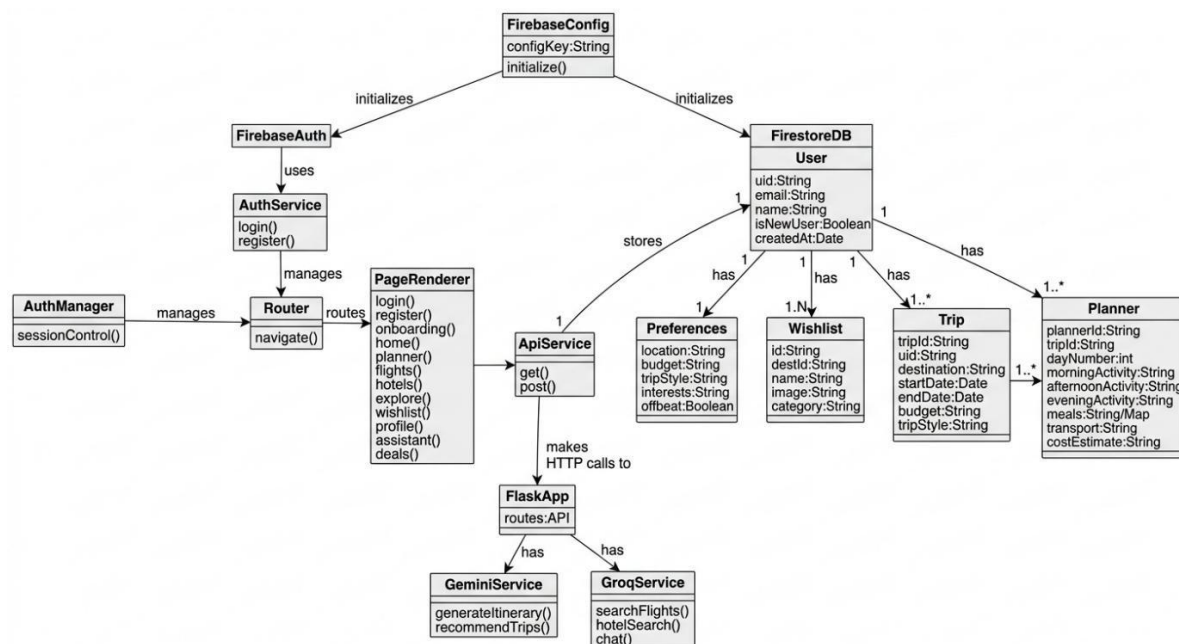
2.4 Table Structure :



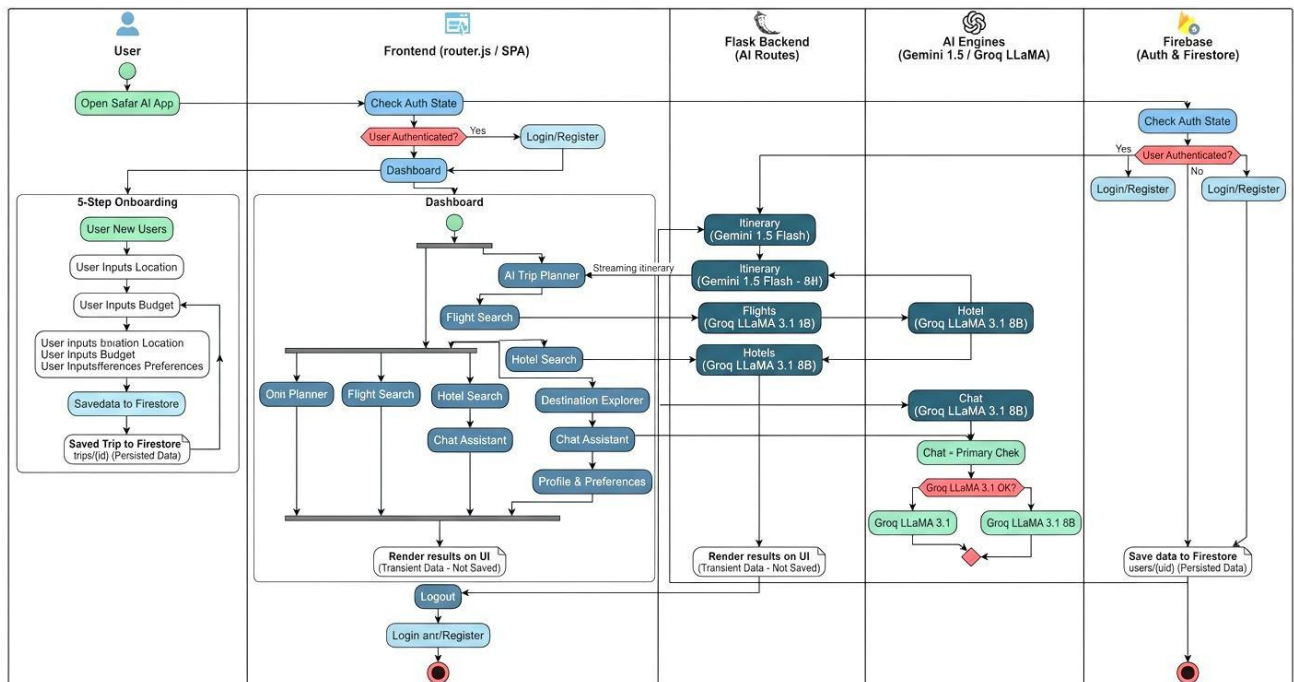
2.5 Use Case Diagram :



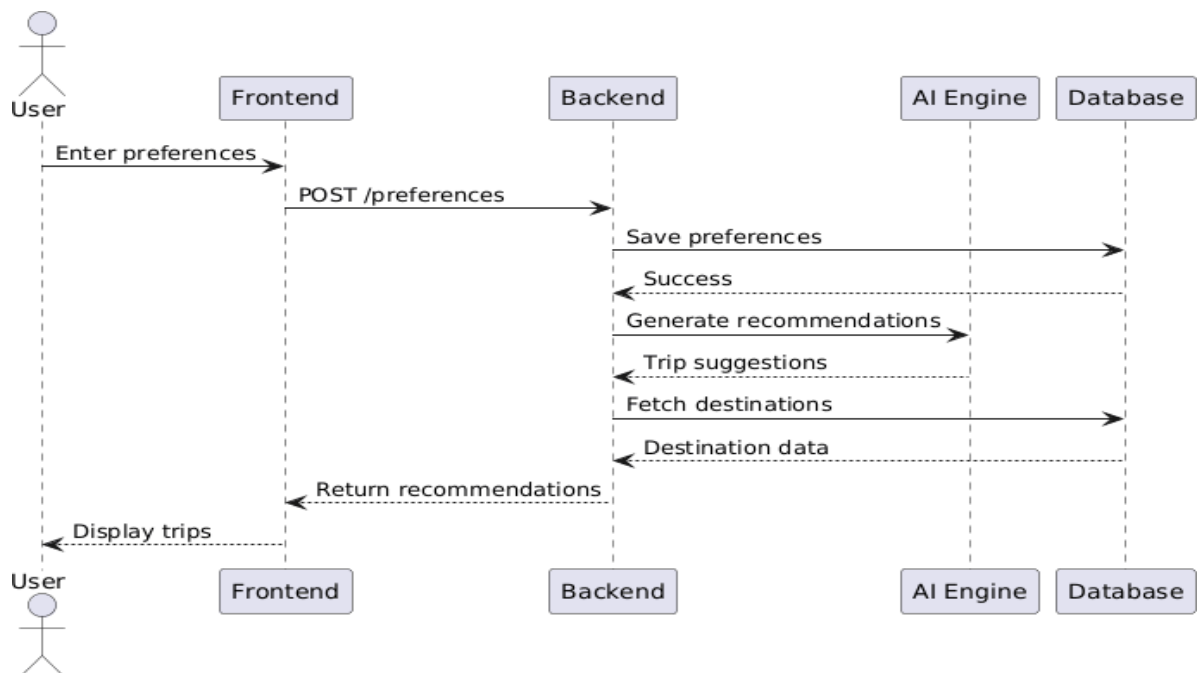
2.6 Class Diagram :



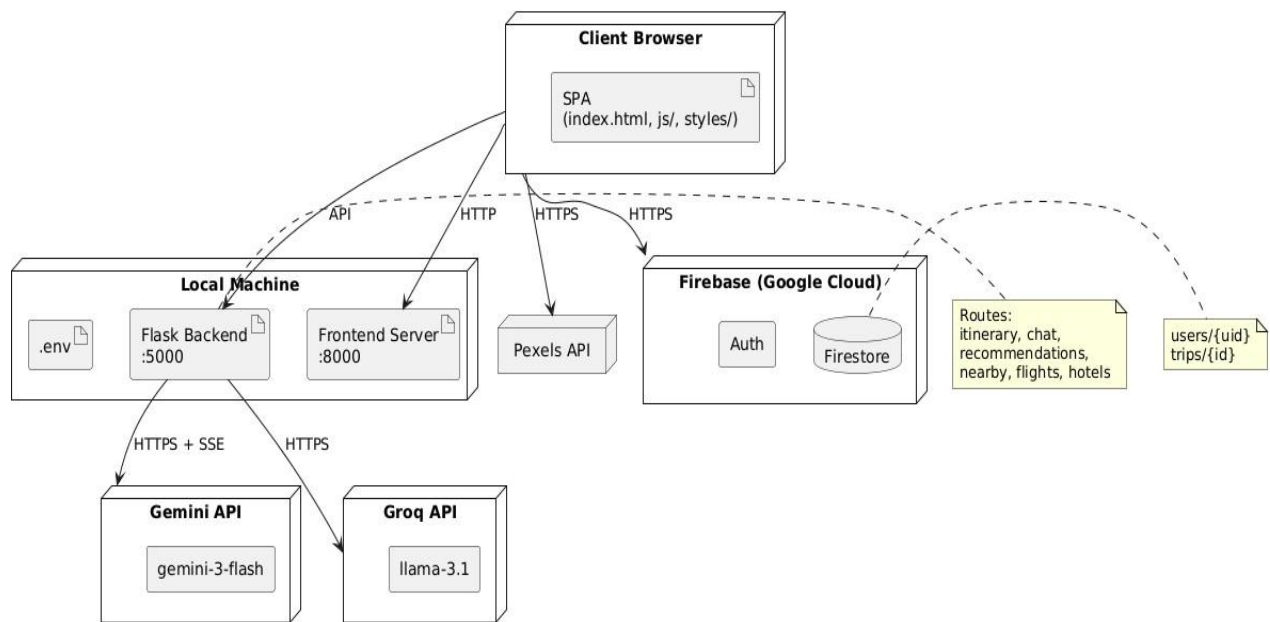
2.7 Activity Diagram :



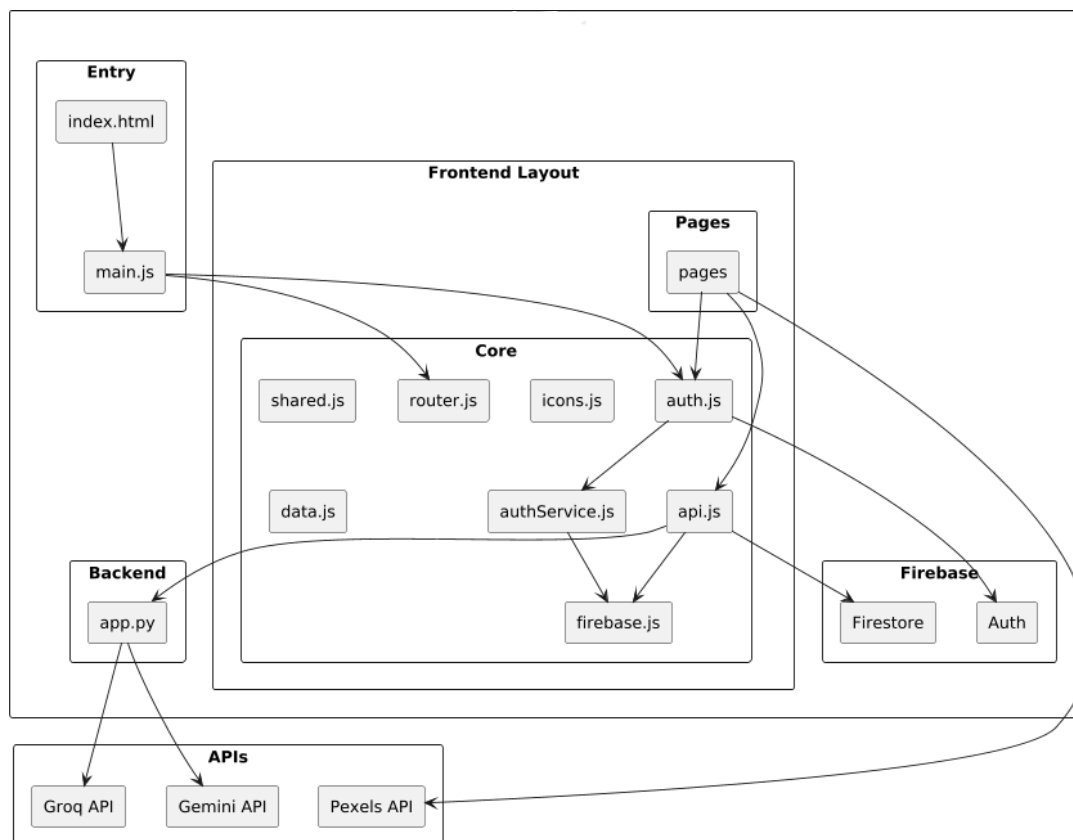
2.8 Sequence Diagram :



2.9 Deployment Diagram :



2.10 Module Hierarchy Diagram :



2.11 Input and Output Screens :

Sign Up Page:



Safar AI

Join thousands of travellers
planning smarter trips with AI

- AI-powered itineraries
- Smart hotel picks
- Flight recommendations
- Personalized for you

Create Account

Start planning your adventures

Full Name

Email Address

Create Password

Confirm Password

Create free account

OR

 Continue with Google

Already have an account? [Sign In](#)

Location Detect Page:

Step 1 of 4

Where are you from?

We'll show trips near you (300–500 km radius)

Detect my location

LIVE LOCATION DETECTED

Wagholi

unnamed road, Pune District, Wagholi - 412207, Maharashtra, India

or choose manually

Select your city

Choose your city...

Continue >

User Preference Page:

Step 2 of 4 < Back

 **What excites you?**
Pick all that interest you

 Historic Places	 Nature & Scenic
 Spiritual & Religious	 Beaches
 Adventure & Trekking	 Food & Culture
 City Life	 Shopping

Continue >

User Trip Budget Page:

Step 3 of 4 < Back

 **Budget comfort**
Used to filter transport, stays, and activities

 Budget Friendly Under ₹5,000
 Balanced ₹5,000 - ₹15,000
 Premium ₹15,000+

Continue >

Trip Style Page:

Step 4 of 4 ← Back

Trip style
How do you like to travel?

☐ Weekend Getaway
 ☒ Explorer
 ☐ Relaxed
 ☐ Backpacking

☒ Offbeat / Less-Crowded
 Hidden gems away from crowds

Start Exploring 🗺️

Home Screen Page:

Hey, Bhushan Jawale 🙌
Where to next? 📍 Wagholi 🔔

AI-POWERED TRAVEL

Plan Your Journey

Personalised itineraries in seconds

Start Planning →

✈️
Flights

🏨
Hotels

🗺️
Explore

🎁
Deals

IN Popular in India
Top destinations to explore



Goa
Goa

₹8k-15k

Beaches Nightlife

4.6



Jaipur
Rajasthan

₹5k-12k

Heritage Culture

4.5



Manali
Himachal Pradesh

₹7k-14k

Adventure Snow

4.5



Varanasi
Uttar Pradesh

₹4k-8k

Spiritual Culture

4.4



Udaipur
Rajasthan

₹6k-13k

Romantic Heritage

4.7

🏠
Home

🔍
Plan

💬
AI Chat

🕒
Help

👤
Profile

3. Coding

3.1 Complete Code of Project or one Module:

Auth.js

```
import { doc, setDoc, getDoc } from 'https://www.gstatic.com/firebasejs/10.12.2/firebase-firestore.js';
import { db, firebaseAuth } from './firebase.js';
import {
  signupUser,
  loginUser,
  loginWithGoogle as fbLoginWithGoogle,
  logoutUser,
  watchAuthState,
  getUserDoc,
} from './authService.js';

class AuthManager {
  constructor() {
    this.user = null;
    this.wishlist = [];
    this.listeners = [];
    this._ready = false;
    this._readyResolvers = [];

    // Single source of truth: Firebase auth state
    watchAuthState(async (firebaseUser) => {
      if (firebaseUser) {
        const data = await getUserDoc(firebaseUser.uid);
        this.user = data ? { uid: firebaseUser.uid, ...data } : { uid: firebaseUser.uid, email:
firebaseUser.email };
        this.wishlist = this.user.wishlist || [];
      } else {
        this.user = null;
        this.wishlist = [];
      }
      this._resolveReady();
      this.notify();
    });
  }
}
```

```

// Resolves all pending getUser() calls once auth state is known
_resolveReady() {
  if (!this._ready) {
    this._ready = true;
    this._readyResolvers.forEach(r => r(this.user));
    this._readyResolvers = [];
  }
}

// Waits for the first onAuthStateChanged to fire before returning
getUser() {
  if (this._ready) return Promise.resolve(this.user);
  return new Promise(resolve => this._readyResolvers.push(resolve));
}

subscribe(listener) { this.listeners.push(listener); }
notify()           { this.listeners.forEach(l => l(this.user)); }

// Auth actions

async signup(name, email, password, confirmPassword = password) {
  const result = await signupUser(name, email, password, confirmPassword);
  return result; // { success, error }
}

async login(email, password) {
  const result = await loginUser(email, password);
  if (!result.success) return { success: false, error: result.error };
  // Wait for onAuthStateChanged to fire and populate this.user
  await new Promise(resolve => {
    const unsub = watchAuthState((u) => { if (u) { unsub(); resolve(); } });
  });
  return { success: true };
}

async loginWithGoogle() {
  const result = await fbLoginWithGoogle();
  if (!result.success) return { success: false };
  // Wait for onAuthStateChanged to populate this.user
  await this.getUser();
  return { success: true, isNewUser: result.isNewUser };
}

// backward compat alias

```

```

async googleLogin() { return this.loginWithGoogle(); }

async logout() {
  await logoutUser();
  this.user = null;
  this.wishlist = [];
  this._ready = true;
  this._readyResolvers = [];
  this.notify();
}

// Profile / Wishlist
async completeOnboarding(preferences) {
  if (!this.user) return;
  await setDoc(doc(db, 'users', this.user.uid), { isNewUser: false, preferences }, { merge:
true });
  this.user.isNewUser = false;
  this.user.preferences = preferences;
  this.notify();
}

async toggleWishlist(id) {
  if (!this.user) return;
  const inList = this.wishlist.includes(id);
  this.wishlist = inList ? this.wishlist.filter(i => i !== id) : [...this.wishlist, id];
  await setDoc(doc(db, 'users', this.user.uid), { wishlist: this.wishlist }, { merge: true });
  this.notify();
}

async loadWishlist() {
  if (!this.user) return;
  const data = await getUserDoc(this.user.uid);
  this.wishlist = data?.wishlist || [];
}

isAuthenticated() { return !!this.user; }
getUserData() { return this.user; }
isInWishlist(id) { return this.wishlist.includes(id); }
}

export const auth = new AuthManager();

```


4. Testing

4.1 Test Strategy

The testing strategy for Safar AI follows a layered approach covering backend API logic, frontend UI behaviour, and end-to-end user flows.

Scope:

- Backend REST API endpoints (Flask/Python)
- Frontend page rendering and user interactions (Vanilla JS)
- AI integration responses (Gemini, Groq)
- Firebase Authentication flows

Approach:

- Unit tests validate individual functions and API endpoints in isolation using mocked AI responses
- Acceptance tests validate complete user journeys from the browser
- All tests are run locally before deployment; no CI/CD pipeline is configured at this stage

Tools:

Layer	Tool
Backend unit tests	Pytest + unittest.mock
Frontend unit tests	Manual / browser console
API integration tests	Pytest + requests
Acceptance tests	Manual browser walkthrough

Out of Scope:

- Load/performance testing
- Cross-browser automated testing
- Real payment or booking flows (app uses AI-generated mock data)

4.2 Unit Test Plan

Backend (Python / Flask)

Test ID	Module	Function Under Test	Input	Expected Output
UT-B01	app.py	gemini()	Valid prompt	Returns non-null string
UT-B02	app.py	gemini()	API returns 429	Retries up to 3 times, returns None on exhaustion
UT-B03	app.py	groq_chat()	Valid messages list	Returns assistant message string
UT-B04	app.py	groq_chat()	Network timeout	Returns None gracefully
UT-B05	app.py	GET /api/recommendations	Valid preferences JSON	Returns JSON with destinations array ≥ 1 item
UT-B06	app.py	GET /api/recommendations	Gemini fails	Returns default recommendations fallback
UT-B07	app.py	GET /api/nearby	{ "location": "Mumbai" }	Returns JSON with destinations array
UT-B08	app.py	GET /api/flights/search	Valid city + date	Returns { success: true, flights: [...] }
UT-B09	app.py	GET /api/hotels/search	Valid city + budget	Returns { success: true, hotels: [...] }
UT-B10	app.py	GET /api/chat	{ message: "Plan a trip to Goa" }	Returns { success: true, response: "..." }
UT-B11	app.py	GET /api/chat	Groq fails	Falls back to Gemini, still returns success
UT-B12	app.py	_default_recommendations()	—	Returns list of 6 hardcoded destinations
UT-B13	app.py	_default_nearby()	—	Returns list of 8 hardcoded destinations

Frontend (JavaScript)

Test ID	Module	Function Under Test	Condition	Expected Behaviour
UT-F01	flights.js	parsePriceNum()	Input "₹4,500"	Returns 4500
UT-F02	flights.js	parsePriceNum()	Input is number 3200	Returns 3200
UT-F03	flights.js	formatDate()	Input "2025-08-15"	Returns "15 Aug 2025"
UT-F04	flights.js	swapCities()	From = "Mumbai", To = "Delhi"	Values are swapped in inputs
UT-F05	planner.js	handleGenerate()	Empty source/destination	Form shakes, no API call made
UT-F06	planner.js	Day counter	Click + 30 times	Value caps at 30
UT-F07	planner.js	Day counter	Click – below 1	Value stays at 1
UT-F08	login.js	Login form submit	Empty email/password	Form does not submit
UT-F09	login.js	Password toggle	Click eye icon	Input type toggles between password and text

4.3 Acceptance Test Plan

Acceptance tests verify complete user-facing scenarios as a real user would experience them.

Preconditions: Backend running on localhost:5000, frontend on localhost:8000, valid API keys in .env.

AT ID	Feature	User Story	Pass Criteria
AT-01	Registration	New user signs up with email/password	Account created, redirected to /onboarding
AT-02	Google Login	User signs in with Google	Authenticated, redirected to /home or /onboarding for new users
AT-03	Onboarding	User completes preference setup	Preferences saved to Firestore, redirected to /home
AT-04	Home Recommendations	User views home page	AI-generated destination cards load within 10 seconds
AT-05	Trip Planner	User enters Mumbai → Goa, 3 days, Balanced	Itinerary with flights, trains, hotels, and daily plan is displayed
AT-06	Trip Planner – Save	User clicks "Save Trip"	Trip saved to Firestore, redirected to /my-trips
AT-07	Flight Search	User searches Mumbai → Delhi, Economy	At least 4 flight results displayed with price, airline, and duration
AT-08	Hotel Search	User searches hotels in Jaipur, mid budget	At least 5 hotel results displayed with name, rating, and price
AT-09	AI Chat	User sends "Best places to visit in Kerala"	Assistant responds with relevant suggestions within 5 seconds
AT-10	Wishlist	User adds a destination to wishlist	Destination appears in /wishlist page
AT-11	Profile	User views profile page	Name, email, and preferences are displayed correctly
AT-12	Logout	User logs out	Session cleared, redirected to /login

4.4 Test Cases / Test Scripts

Backend Test Script (backend/tests/test_app.py)

```
import pytest
from unittest.mock import patch, MagicMock
import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..'))
from app import app, _default_recommendations, _default_nearby

@pytest.fixture
def client():
    app.config['TESTING'] = True
    with app.test_client() as c:
        yield c

# — Default fallback functions —————

def test_default_recommendations_count():
    assert len(_default_recommendations()) == 6

def test_default_nearby_count():
    assert len(_default_nearby()) == 8

def test_default_recommendations_fields():
    item = _default_recommendations()[0]
    assert all(k in item for k in ['name', 'state', 'distance', 'budget', 'tags', 'rating'])

# — /api/recommendations —————

def test_recommendations_returns_fallback_when_gemini_fails(client):
    with patch('app.gemini', return_value=None):
        res = client.post('/api/recommendations', json={'location': 'Mumbai'})
        assert res.status_code == 200
        data = res.get_json()
        assert 'destinations' in data
        assert len(data['destinations']) > 0
```

```
def test_recommendations_parses_gemini_response(client):
    mock_json =
    '[{"name": "Goa", "state": "Goa", "distance": 450, "budget": "₹8000", "duration": "3
    days", "tags": ["beach"], "rating": 4.7, "reason": "Great beach"}]'
    with patch('app.gemini', return_value=mock_json):
        res = client.post('/api/recommendations', json={'location': 'Mumbai'})
        assert res.status_code == 200
        assert res.get_json()['destinations'][0]['name'] == 'Goa'
```

— */api/nearby* —————

```
def test_nearby_returns_fallback_when_gemini_fails(client):
    with patch('app.gemini', return_value=None):
        res = client.post('/api/nearby', json={'location': 'Pune'})
        assert res.status_code == 200
        assert len(res.get_json()['destinations']) > 0
```

— */api/flights/search* —————

```
def test_flight_search_success(client):
    mock_flights = '[{"airline": "IndiGo", "flight_number": "6E-
    101", "departure_time": "06:00", "arrival_time": "08:00", "from_code": "BOM", "to
    _code": "DEL", "duration": "2h", "stops": 0, "cabin_class": "Economy", "price": "₹45
    00", "amenities": ["Snacks"]}]'
    with patch('app.groq_chat', return_value=mock_flights):
        res = client.post('/api/flights/search', json={'from': 'Mumbai', 'to': 'Delhi',
        'date': '2025-08-15', 'passengers': 1, 'cabin_class': 'economy'})
        assert res.status_code == 200
        data = res.get_json()
        assert data['success'] is True
        assert len(data['flights']) == 1
```

```
def test_flight_search_groq_failure(client):
    with patch('app.groq_chat', return_value=None):
        res = client.post('/api/flights/search', json={'from': 'Mumbai', 'to': 'Delhi',
        'date': '2025-08-15'})
        assert res.status_code == 500
```

— */api/hotels/search* —————

```
def test_hotel_search_success(client):
    mock_hotels = '[{"name": "Taj
Hotel", "type": "Luxury", "location": "Colaba", "rating": 4.8, "price_per_night": "₹80
00", "amenities": ["Pool", "Spa"], "highlight": "Sea view"}]'
    with patch('app.groq_chat', return_value=mock_hotels):
        res = client.post('/api/hotels/search', json={'city': 'Mumbai', 'budget':
'luxury', 'guests': 2, 'rooms': 1})
        assert res.status_code == 200
        assert res.get_json()['success'] is True
```

— */api/chat* —————

```
def test_chat_success_via_groq(client):
    with patch('app.groq_chat', return_value='Kerala is beautiful! Visit Munnar
and Alleppey.'):
        res = client.post('/api/chat', json={'message': 'Best places in Kerala?'})
        assert res.status_code == 200
        assert res.get_json()['success'] is True
```

```
def test_chat_falls_back_to_gemini(client):
    with patch('app.groq_chat', return_value=None), \
        patch('app.gemini', return_value='Try Munnar for hills.'):
        res = client.post('/api/chat', json={'message': 'Hill stations?'})
        assert res.status_code == 200
        assert 'response' in res.get_json()
```

```
def test_chat_fails_when_both_apis_down(client):
    with patch('app.groq_chat', return_value=None), \
        patch('app.gemini', return_value=None):
        res = client.post('/api/chat', json={'message': 'Hello'})
        assert res.status_code == 500
```

Copypython

Frontend Unit Test Script (Browser Console)

// Run in browser DevTools console on http://localhost:8000

// UT-F01: parsePriceNum with currency string

```
import('/js/pages/flights.js').then(m => {
  // parsePriceNum is not exported — test via renderFlightResults sort
  behaviour
  console.assert(parseInt("₹4,500".replace(/^[^0-9]/g, "")) === 4500, 'UT-F01
  PASS');
});
```

// UT-F03: formatDate

```
const formatDate = (d) => new Date(d).toLocaleDateString('en-IN', { day:
'numeric', month: 'short', year: 'numeric' });
console.assert(formatDate('2025-08-15').includes('Aug'), 'UT-F03 PASS');
```

// UT-F06 / UT-F07: Day counter bounds

// Navigate to /planner, then run:

```
document.getElementById('days').value = 30;
document.getElementById('incDay').click();
console.assert(parseInt(document.getElementById('days').value) === 30, 'UT-
F06 cap at 30 PASS');
```

```
document.getElementById('days').value = 1;
document.getElementById('decDay').click();
console.assert(parseInt(document.getElementById('days').value) === 1, 'UT-
F07 floor at 1 PASS');
```

Copyjavascript

Running Backend Tests

```
cd backend
```

```
pip install pytest
```

```
pytest tests/test_app.py -v
```

```
Copybash
```

Expected output:

test_default_recommendations_count	PASSED
test_default_nearby_count	PASSED
test_default_recommendations_fields	PASSED
test_recommendations_returns_fallback	PASSED
test_recommendations_parsing_gemini	PASSED
test_nearby_returns_fallback	PASSED
test_flight_search_success	PASSED
test_flight_search_groq_failure	PASSED
test_hotel_search_success	PASSED
test_chat_success_via_groq	PASSED
test_chat_falls_back_to_gemini	PASSED
test_chat_fails_when_both_apis_down	PASSED

5. Limitations of the Proposed System

5.1 AI-Generated Data is Not Real-Time

All flight, hotel, train, and cab data displayed in Safar AI is generated by the Groq (LLaMA) and Google Gemini language models. The data is plausible and realistic but is **not sourced from live booking systems**. Flight prices, availability, train schedules, and hotel rates are illustrative only and cannot be used to make actual bookings. The system has no integration with GDS (Global Distribution Systems) such as Amadeus, Sabre, or IRCTC.

5.2 No Real Booking or Payment Capability

The platform currently has no payment gateway integration (e.g., Razorpay, Stripe). The "Book Now" buttons on flight and hotel cards are UI placeholders. Users cannot complete an actual reservation from within the application.

5.3 India-Only Scope

The recommendation engine, nearby destinations, and itinerary generator are explicitly scoped to Indian travel destinations. The system prompt and default fallback data are hardcoded for India. International travel planning is not supported.

5.4 API Rate Limits and Latency

The system depends on two third-party AI APIs:

- **Gemini API** — subject to quota limits; the backend retries on HTTP 429 with exponential backoff (up to 3 attempts), but sustained high usage will result in failures falling back to static default data.
- **Groq API** — has a 20-second timeout and a

max_tokens

cap of 1024, which can truncate responses for complex queries.

Itinerary generation via streaming can take 15–60 seconds depending on network conditions and API load.

5.5 No Offline Support

The application requires an active internet connection for all AI features, Firebase Authentication, and Firestore data access. There is no service worker, caching strategy, or offline fallback beyond the hardcoded default destination lists.

5.6 Single-User Session, No Collaboration

Trip planning is tied to a single authenticated user. There is no feature to share a trip, collaborate on an itinerary with other users, or export a trip plan to PDF or calendar formats.

5.7 Hardcoded API Base URL

The frontend API base URL is hardcoded as

`http://localhost:5000/api`

in both `[api.js](c:\Users\bhush\OneDrive\Desktop\Safar AI Project\js\api.js#L4)` and `[flights.js](c:\Users\bhush\OneDrive\Desktop\Safar AI Project\js\pages\flights.js#L5)`. This makes deployment to a production environment require manual code changes rather than environment-based configuration.

5.8 No Input Validation on the Backend

The Flask backend performs no server-side validation or sanitisation on incoming request fields (e.g., source, destination, city). Malformed or adversarial inputs are passed directly into AI prompts, which could produce unexpected or irrelevant outputs.

5.9 Chat History Not Persisted

The AI chat assistant accepts a history array per request but this history is managed only in the browser session. Conversation history is lost on page refresh and is never saved to Firestore, so users cannot resume previous conversations.

6. Proposed Enhancements

6.1 Live Flight and Hotel Data Integration

Replace AI-generated travel data with real-time data from:

- **Flights** — Amadeus Flight Offers Search API or Skyscanner Rapid API
- **Hotels** — MakeMyTrip API or Booking.com Affiliate API

This would enable actual availability checks, live pricing, and real booking flows.

6.2 Payment Gateway Integration

Integrate **Razorpay** (preferred for Indian market) to enable end-to-end booking and payment within the platform. This would transform Safar AI from a planning tool into a full travel commerce platform.

6.3 Expand to International Destinations

Remove the India-only constraint from AI prompts and default data. Add currency conversion support and international airline/hotel data sources to support global trip planning.

6.4 Progressive Web App (PWA) with Offline Support

Convert the frontend into a PWA by adding:

- A service worker for caching static assets and last-viewed itineraries
- A web app manifest for home screen installation
- IndexedDB storage for offline access to saved trips

6.5 Trip Sharing and Collaboration

Allow users to share a trip plan via a unique link. Implement real-time collaborative editing using Firestore's onSnapshot listener so multiple users (e.g., travel companions) can view and edit the same itinerary simultaneously.

6.6 PDF and Calendar Export

Add the ability to export a generated itinerary as:

- A formatted **PDF** using a library such as jsPDF
- An **ICS calendar file** so activities can be imported into Google Calendar or Apple Calendar

6.7 Environment-Based Configuration

Replace hardcoded API URLs with a build-time environment configuration system. For the current Vanilla JS stack, this can be achieved using a config.js file that is swapped at deployment, or by migrating to a bundler (Vite/Webpack) that supports .env files natively on the frontend.

6.8 Backend Input Validation and Rate Limiting

Add server-side validation using a library such as Marshmallow or pydantic to sanitise all incoming API request fields. Implement per-IP rate limiting using Flask-Limiter to prevent abuse of the AI endpoints.

6.9 Persistent Chat History

Save chat conversation history to Firestore under users/{uid}/chats/{chatId} so users can resume previous conversations. Display a chat history sidebar in the assistant page.

6.10 Multi-Language Support

Add internationalisation (i18n) support for Hindi and other regional Indian languages, making the platform accessible to a broader demographic. The AI chat endpoint already supports multi-language responses from Groq/Gemini natively.

7. Conclusion

Safar AI is an AI-powered travel planning platform designed to simplify and personalise the end-to-end trip planning experience for Indian travellers. The system integrates Google Gemini and Groq (LLaMA 3.1) large language models with a lightweight Flask backend and a Vanilla JavaScript single-page application frontend, backed by Firebase for authentication and data persistence.

The platform successfully delivers its core objectives:

- Personalised destination recommendations based on user preferences, location, budget, and travel style
- AI-generated, multi-day itineraries with transport, accommodation, and daily activity breakdowns
- Real-time flight and hotel search powered by conversational AI
- A context-aware travel assistant chatbot with Groq-to-Gemini fallback resilience
- Secure user authentication via Firebase (email/password and Google OAuth)
- Wishlist management and trip saving to Firestore

The architecture demonstrates a practical and cost-effective approach to building AI-integrated web applications — using free-tier AI APIs, serverless Firebase infrastructure, and a zero-dependency frontend to deliver a feature-rich product without significant infrastructure overhead.

While the current system has clear limitations — most notably the absence of real-time booking data and live payment processing — these are well-understood constraints of the prototype scope. The proposed enhancements in Section 6 provide a clear and actionable roadmap to evolve Safar AI from a planning and discovery tool into a fully transactional travel platform.

Safar AI demonstrates that modern large language models, when combined with thoughtful UX design and a structured backend, can meaningfully reduce the complexity of travel planning and deliver a genuinely useful, personalised experience to users.

8. Bibliography

1. Amazon Web Services. (2024). Amazon Q Developer — AI-Powered Code Assistant. AWS Developer Tools. <https://aws.amazon.com/q/developer/> (Used as an AI coding assistant during the development of Safar AI for code generation, debugging, and documentation.)
2. Google. (2024). *Gemini API Documentation — generateContent & streamGenerateContent*. Google AI for Developers. <https://ai.google.dev/api/generate-content>
3. Groq Inc. (2024). *Groq API Reference — Chat Completions*. Groq Cloud Documentation. <https://console.groq.com/docs/openai>
4. Meta AI. (2024). *LLaMA 3.1 Model Card*. Meta AI Research. <https://ai.meta.com/blog/meta-llama-3-1/>
5. Google Firebase. (2024). *Firebase Authentication Documentation*. Firebase by Google. <https://firebase.google.com/docs/auth>
6. Google Firebase. (2024). *Cloud Firestore Documentation*. Firebase by Google. <https://firebase.google.com/docs/firestore>
7. Pallets Projects. (2024). *Flask Documentation (v3.0)*. <https://flask.palletsprojects.com/en/3.0.x/>
8. Flask-CORS Contributors. (2024). *Flask-CORS Documentation*. <https://flask-cors.readthedocs.io/en/latest/>
9. Python Software Foundation. (2024). *Python 3 Standard Library — json, re, os, time*. <https://docs.python.org/3/library/>
10. MDN Web Docs. (2024). *Fetch API — Using Fetch*. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
11. Pytest Development Team. (2024). *pytest Documentation*. <https://docs.pytest.org/en/stable/>
12. Ministry of Tourism, Government of India. (2023). *India Tourism Statistics 2023*. <https://tourism.gov.in/india-tourism-statistics>
13. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
14. Pexels. (2024). Pexels API Documentation — Photo Search Endpoint. Pexels for Developers. <https://www.pexels.com/api/documentation/> (Used in shared.js to dynamically fetch high-quality destination photographs for the Home and Explore pages based on destination search queries.)